Startup-Centric Benchmarking of Free-Tier Cloud-Managed Databases

Avyaansh Dixit[a,*] (Researcher), Nidhi Singh[b,**] (Researcher), Deepinder Kaur[a] (Researcher), Aryan[a] (Researcher) and Gargi Singh[c] (Researcher)

[a]*KIET Group of Institutions, Department of Computer Science and Engineering (AI), 13-KM Stone, Delhi-Meerut Road, Ghaziabad, 201206, Uttar Pradesh, India*

[b]*KIET Group of Institutions, Department of Computer Science and Engineering (AI&ML), 13-KM Stone, Delhi-Meerut Road, Ghaziabad, 201206, Uttar Pradesh, India*

[c]*Christ University (Deemed to be University), School of Science, Nandgram Rd, Marium Nagar, Sewa Nagar, Ghaziabad, 201003, Uttar Pradesh, India*

ARTICLE INFO

Keywords:
Database benchmarking
Cloud databases
E-commerce applications
Performance evaluation
Free-tier platforms
Startup technology selection

ABSTRACT

Choosing the right database architecture is a pivotal decision for early-stage e-commerce startups, especially under resource and expertise constraints. Existing benchmarking research disproportionately evaluates enterprise-grade solutions, leaving a gap for evidence relevant to startup-accessible free-tier cloud platforms. This paper introduces a reproducible framework for comparing leading cloud-hosted database systems, employing realistic e-commerce scenarios that reflect operational priorities of growing digital ventures. The evaluation encompasses multiple workload types—targeting read, write, and mixed operation phases—while utilizing modular strategies for index optimization and query design. Emphasis is placed on capturing both performance consistency and operational reliability, with a methodology enabling transparent metric computation and result visualization. The study highlights meaningful differences in how free-tier cloud databases manage diverse transactional demands, revealing nuanced trade-offs integral to platform selection at the startup stage. Practitioners and researchers will find actionable guidance that transforms database choices from trial-and-error decisions into a systematic, evidence-based process.

Highlights

- First free-tier cloud database benchmark for early-stage e-commerce startups
- MongoDB achieved 2.6× faster throughput than PostgreSQL in mixed workloads
- Supabase demonstrated 97.68% cache effectiveness on aggregation queries
- AstraDB exhibited 413-second failures on partition-misaligned aggregations
- Reproducible framework democratizes performance evaluation for startups

1. Introduction

India's e-commerce is booming, projected to reach US $ 350 billion by 2030 and become the second-largest market by 2034, driven by mobile use, UPI, payments, and initiatives such as Digital India and ONDC. Mobile commerce will account for nearly 73% of sales. Rural and semi-urban areas are growth zones but face challenges such as illiteracy, language barriers, and trust issues. Despite more than 157,000 startups, e-commerce founders struggle with database choices due to limited technical expertise. Benchmarking studies focus on enterprise systems that lack the constraints of startups, such as cloud limits and evolving data needs. Standard benchmarks like TPC-H and YCSB do not reflect the behavior of e-commerce apps or low-latency demands.

Technical and social challenges, such as language barriers in seller support, financial exclusion, and policies such as ODOP, complicate platform adoption. Technically, databases have trade-offs: MongoDB's 16MB document limit, MyRocks' latency with large records, and free-tier cloud resource caps require careful planning. Our study presents a benchmarking framework for startups, comparing Supabase (PostgreSQL), MongoDB Atlas, and DataStax AstraDB with real e-commerce workloads (browsing, cart management, order, and transactions). We measure latencies (p50, p95, p99), throughput, and performance within free tiers to guide database choices. The paper bridges market, socio-technical, and system evaluations, offering practical insights for Indian e-commerce startups.

India's e-commerce sector is one of the fastest-growing markets in the world, expected to reach $ 350 billion by 2030 and possibly overtake the US as the second-largest by 2034 (Thete, 2024). Key drivers include widespread smartphone use, digital payment systems like UPI, and government initiatives such as Digital India and ONDC (Thete, 2024)(Akther, 2023). However, socio-technical hurdles persist. High illiteracy rates (43%, increasing to 70–80% in northern states), low digital literacy, and language gaps hinder adoption (Akther, 2023)(Bhatia-Kalluri, 2021). Rural and semi-urban entrepreneurs, which comprise two-thirds of the population, face issues such as mistrust in

*Corresponding author
**Corresponding author
Email addresses: avyaansh.2226cseai10@kiet.edu (A. Dixit); nidhi.singh@kiet.edu (N. Singh); deepinder.kaur@kiet.edu (D. Kaur); aryan.2226ec1096@kiet.edu (Aryan); gargi.singh@christuniversity.in (G. Singh)
ORCID(s): 0009-0009-0533-8418 (A. Dixit); 0009-0003-0858-1743 (N. Singh); 0009-0004-3249-0785 (D. Kaur); 0009-0005-6185-5541 (Aryan); 0009-0008-7481-4628 (G. Singh)

digital payments and limited local language support (Bhatia-Kalluri, 2021). Government schemes like ODOP and partnerships with e-commerce giants aim to address these gaps, but inclusivity, accessibility, and trust remain challenges.

Database layers are vital for e-commerce, affecting performance, scalability, and sustainability. Relational databases like PostgreSQL offer structure and consistency, while systems like MongoDB provide flexibility for semi-structured data. Wide-Column Store (or Columnar) databases like AstraDB excel at relationship-centric tasks like recommendations. Challenges include MongoDB's 16MB document size limit and delays in storage engines such as MyRocks with extensive records, which affects scalability and read-heavy workloads (Liyanage, Pandey, Goldstein, Cahill, Dey, Fekete and Röhm, 2025). As e-commerce needs evolve with frequent reads, writes, and dynamic data, choosing suitable databases is crucial. Cloud-managed free-tier options like Supabase, MongoDB Atlas, and DataStax AstraDB increase accessibility but come with resource limits, creating trade-offs, particularly for startups balancing costs and growth.

Database benchmarking has long been used to assess system performance, with standards such as TPC-H, TPC-DS, and YCSB measuring throughput, transactions, and scalability. These are mainly for large enterprises with large resources. Recent studies highlight limitations for startups: synthetic workloads do not capture real, unpredictable queries on e-commerce platforms, and the impact of data growth remains underexplored. Interactive performance needs are often ignored, despite delays of just one second that make platforms unresponsive. Assessments focus on infrastructure metrics like CPU and disk I/O, rather than application-visible metrics like latency and throughput. Battle et al. (Battle, Eichmann, Angelini, Catarci, Santucci, Zheng et al., 2020) state that no DBMS performs well with datasets larger than 10M rows during startup growth. Liyanage et al. (Liyanage et al., 2025) also show that key/value and document databases have variable value lengths, often overlooked in benchmarks. Recent studies of cloud databases (TiDB Team, 2025) reveal that managing data growth in limited environments requires careful trade-off analysis. However, few systematic studies focus on startup constraints, especially in Indian e-commerce.

Although current research provides insight into India's e-commerce, database trade-offs, and benchmarking, there is a gap in startup-specific evaluations. These should consider free-tier cloud constraints, realistic workloads with reads and writes, and application-visible metrics like latencies and throughput. Such benchmarks can help founders make better database decisions.

2. Literature Review

Benchmarking of database systems has evolved significantly in the past two decades, driven by the emergence of diverse database paradigms, cloud computing platforms, and increasingly complex application requirements. This review of the literature examines the current state of database benchmarking research in five critical dimensions: database types and hosting models, data set characteristics, performance metrics, methodological approaches, and research motivations. Through systematic analysis of existing work, we identify substantial gaps in startup-focused, resource-constrained evaluation frameworks that justify our novel approach to free-tier database benchmarking.

2.1. Database Types, Hosting Models, and Service Tiers

2.1.1. Enterprise-Scale Dominance

The landscape of database benchmarking has been dominated by enterprise-scale evaluations that largely ignore the practical constraints faced by startups and resource-limited organizations. The LDBC Social Network Benchmark: Business Intelligence Workload represents the current state-of-the-art in graph database evaluation, testing systems such as TigerGraph and Umbra on high-performance AWS and GCP instances with substantial computational resources (Szárnyas, Waudby, Steer, Szakállas, Birler, Wu et al., 2022). However, this enterprise-only focus creates a critical gap, as the complete absence of resource-constrained or cost-optimized testing leaves startup scenarios entirely unaddressed. Similarly, the original LDBC Social Network Benchmark: Interactive Workload, while establishing important methodological foundations, conducted all experiments on enterprise-grade hardware with 192GB RAM and dual Xeon processors (Erling, Averbuch, Larriba-Pey, Chafi, Gubichev, Prat et al., 2015). This approach, though valuable for system comparison, fails to reflect the reality that most developers operate under significant budget and infrastructure limitations.

2.1.2. Resource Constraint Gap

Recent comprehensive analysis confirms this enterprise bias, noting that "35 of the 49 audits conducted so far were performed in the cloud" with significant costs (Szárnyas, Bebee, Birler, Deutsch, Fletcher, Gabb et al., 2023). This acknowledgment that "prohibitively expensive" cloud egress fees, combined with an enterprise-only evaluation framework, reveal a gap between benchmarking research and real-world database selection requirements. This pattern is seen in various database paradigms. Although these studies offer useful insights into graph database performance, they typically assume unlimited resources, which does not reflect the reality for most developers and organizations who need to balance performance needs with limited budgets.

2.2. Dataset Characteristics and Preprocessing

2.2.1. Synthetic vs. Realistic Data Challenge

The current benchmarking literature reveals a core tension between the use of synthetic data generation methods and the requirement for realistic application scenarios. The influential YCSB framework prioritized simplicity over realism, explicitly stating: "We did not attempt to exactly model a particular application or set of applications" (Cooper, Silberstein, Tam, Ramakrishnan and Sears, 2010). Although this generic approach allowed compatibility between various

systems, it also resulted in a notable shortfall in application-specific assessments. The YCSB++ extension was intended to overcome some limitations by adding features such as bulk loading and server-side filtering (Patil, Polte, Ren, Tantisiriroj, Xiao, López et al., 2011). However, it continued to depend on generic key-value workloads rather than actual application behavior. Although these improvements aid system tuning, they do not capture the intricate data relationships and access patterns common in real-world applications.

2.2.2. Domain-Specific Sophistication

The LDBC benchmarks feature advanced synthetic data methods. The SNB Interactive v2 benchmark demonstrates high complexity with a 3-year simulation using real social network patterns, including flash mob events and cascading deletes, reflecting real phenomena (Püroja, Waudby, Boncz and Szárnyas, 2023). Its focus is limited to social networks, neglecting other areas like e-commerce. The TPC-H specification, using DBGEN, employs deterministic rules that can miss the variability of real-world data(Transaction Processing Performance Council, 2014). These frameworks do not address e-commerce challenges, such as changes in product catalogs due to reviews, pricing, or inventory updates, which current datasets do not cover.

2.3. Performance Metrics and Measurement Approaches

2.3.1. Average-Based Limitations

A key limitation in current benchmarking studies is the focus on average performance metrics instead of tail latency features that affect user experience. TPC benchmarking standards, which include metrics such as QphH@Size, depend greatly on geometric means, masking performance outliers. As highlighted in a detailed TPC-H review, this implies that "for end-users, optimizing long-running queries is generally more important" (Boncz, Neumann and Erling, 2013). The LDBC benchmarks perpetuate this limitation despite their sophisticated workload design. The SNB Business Intelligence benchmark explicitly follows "TPC-H in using a geometric mean, ensuring that there is an incentive to improve all queries, no matter their running time" (Szárnyas et al., 2022), yet this approach masks the tail latencies that most directly impact user satisfaction. Even more concerning, while the benchmark collects percentile data, detailed percentile analysis is not systematically reported across all queries.

2.3.2. Infrastructure vs. Application Focus

This infrastructure-centric measurement approach is widely applied in the literature. The LDBC Financial Benchmark, despite its focus on real-time financial applications that require sub-100 ms response times, reports only average execution times and binary timeout thresholds, rather than the percentile distributions essential for SLA compliance (Qi, Tong, Hu, Lin, Pang, Yuan et al., 2025). The interactive query benchmark approaches addresses this gap by focusing on user-experienced latency (Battle et al., 2020). However, even this work primarily reports average response rates rather than comprehensive percentile analysis. Furthermore, existing benchmarks typically combine read and write performance into a single metric, making it difficult to optimize them separately, which is an essential requirement for modern applications. The decision to distinguish between analytical queries and transactional operations is a core architectural choice that current benchmarks do not adequately evaluate.

2.4. Benchmarking Methodologies and Tool Frameworks

2.4.1. Sophisticated but Inaccessible

Benchmarking methods improve understanding of reproducibility and system assessment, but gaps remain in accessibility and deployment. The YCSB framework provides guidelines for extensible benchmarking, with 'complete framework availability' and separation of workload and setup (Cooper et al., 2010). However, it focuses on infrastructure metrics, overlooking application-visible performance. The LDBC ecosystem is the most comprehensive for database assessment. Its Interactive Workload introduces ideas like parameter curation, a'sophisticated approach to ensure consistent performance' that addresses runtime variability (Erling et al., 2015). The Graphalytics benchmark adds rigor by using deterministic algorithms, ensuring that they are validated by comparing them to the reference output (Iosup, Musaafir, Uta, Pérez, Szárnyas, Chafi et al., 2020).

2.4.2. Accessibility Barriers

Despite these developments, a key challenge is the accessibility of these advanced frameworks. The extensive fault injection method showcases sophisticated techniques for evaluating distributed databases, but requires significant infrastructure and expertise, preventing widespread adoption (Ruben, 2021). Although the "modular API design" and the "system-agnostic approach" offer theoretical flexibility, practical obstacles remain substantial. In particular, current methodological frameworks typically assume unlimited resources and enterprise-level infrastructure. Although their advanced parameter curation and validation processes are beneficial for enterprise assessments, they might be too complicated for resource-limited organizations that need database selection guidance.

2.5. Research Motivation and Practical Applications

2.5.1. Enterprise-Centric Focus

Existing benchmarking research on motivation shows a clear pattern: it focuses on enterprise-level challenges but often overlooks the practical constraints that influence database choices for most developers and organizations. Although research identifies real performance bottlenecks in production visualization systems through interactive querying research, evaluation setup (20GB RAM, 12 cores, 30TB storage) requires substantial resources beyond what most startups can afford (Battle et al., 2020). Research explores the challenge of varying value sizes, noting that "popular

records can reach megabyte sizes while others remain small, creating performance disparities" (Liyanage et al., 2025). This research addresses real-world problems faced by companies such as Dgraph Labs. However, since the testing was performed only on enterprise-grade AWS instances, its practical usefulness for resource-limited organizations is limited.

2.5.2. Industry Collaboration Barriers

The LDBC Financial Benchmark exemplifies this enterprise focus, addressing "special features and patterns in the financial industry" through collaboration with major financial technology companies (Qi et al., 2025). While valuable for large financial institutions, the benchmark's infrastructure requirements and certification barriers (3,000 GBP auditing fees) effectively exclude fintech startups and smaller financial service providers. This pattern reveals a fundamental gap in benchmarking research: while technical sophistication has advanced considerably, the practical needs of resource-constrained organizations building real-world applications remain largely unaddressed. The consistent focus on unlimited resource scenarios, enterprise hardware assumptions, and high infrastructure barriers creates opportunities for research that democratizes database performance evaluation.

2.6. Research Gaps and Implications

The comprehensive analysis of existing literature reveals three critical gaps that justify our focus on startup-oriented, free-tier database benchmarking:

Gap 1: Resource-Constrained Evaluation. Existing benchmarks consistently operate under enterprise resource assumptions, with no systematic evaluation of database performance under the budget and infrastructure constraints that characterize startup operations. The absence of free-tier analysis across the major benchmarking frameworks represents a significant opportunity for practical research.

Gap 2: Application-Visible Performance Metrics. Current benchmarks focus on infrastructure-level measurements while ignoring the application-layer performance characteristics that users actually experience. The predominant use of average-based metrics rather than percentile distributions masks the tail latencies that most directly impact user satisfaction and service reliability.

Gap 3: Domain-Specific Workload Patterns. While sophisticated synthetic data generation exists for social networks and financial scenarios, the e-commerce domain—critical for many startup applications—remains largely unaddressed. The lack of realistic e-commerce query patterns, product catalog evolution, and customer behavior modeling represents a substantial gap in practical benchmarking guidance.

2.7. Research Contribution Justification

These gaps collectively justify our research contribution: a comprehensive evaluation of free-tier cloud databases using realistic e-commerce workloads with emphasis on percentile-based performance metrics and application-visible latency characteristics. By addressing these limitations, our work provides practical database selection guidance for the resource-constrained scenarios that characterize the majority of real-world database deployment decisions. The literature review reveals a mature but narrowly focused field of database benchmarking research. Although significant advances have been made in methodological sophistication, synthetic data generation, and system evaluation frameworks, there is a critical gap in addressing the practical needs of resource-constrained organizations. The consistent enterprise focus across major benchmarking initiatives has created an opportunity for research that democratizes database performance evaluation and provides actionable guidance for startup database selection decisions. Our proposed research addresses these gaps by focusing on free-tier database evaluation with realistic e-commerce workloads, emphasizing application-visible performance metrics that directly impact user experience. This approach fills a significant void in the current literature while maintaining methodological rigor established by existing benchmarking frameworks.

Table 1
Summary of Literature Analysis

Paper	Authors	Yr	DB Types	Contrib.	Resources	Rel.
LDBC SNB: BI	Szárnyas et al.	'22	Graph, Rel.	Graph analytics	Enterprise (AWS)	Low
LDBC SNB: Inter.	Erling et al.	'15	Graph, Hybrid	Param. curation	Enterprise (192GB)	Low
LDBC Council	Szárnyas et al.	'24	Multi-graph	Industry std.	Enterprise	Low
YCSB	Cooper et al.	'10	NoSQL	Cloud bench.	Mixed	Med
YCSB++	Patil et al.	'11	Table stores	Feature test	Mixed	Med
LDBC SNB v2	Püroja et al.	'23	Graph	GDPR comply	Enterprise (SF30K)	Low
TPC-H	TPC	'14	Relational	Analytical	Enterprise (100TB)	Low
TPC-H Analyzed	Boncz et al.	'13-'14	Relational	Choke point	Enterprise	Low
Interactive Query	Battle et al.	'20	RDBMS	User-trace	Enterprise (20GB)	Low
Varying Values	Liyanage et al.	'25	MongoDB, Maria	Record size	Enterprise (AWS)	Med
LDBC Financial	Qi et al.	'25	Graph	Financial	Enterprise	Low
Distributed Perf.	BES	'21	CockroachDB	Fault inject	Enterprise (GKE)	Med
Graphalytics	Iosup et al.	'23	Graph proc.	Algorithm	Enterprise	Low

3. Methodology

This methodology follows a systematic approach to evaluate three cloud database platforms—MongoDB Atlas, Supabase/PostgreSQL, and DataStax Astra DB—under realistic early-stage e-commerce workloads. The evaluation framework comprises nine interconnected components designed to ensure a comparable and reproducible performance assessment in distinct database paradigms.

The objective and Configuration establish the study's goals and technical environment, defining measurement targets and tooling constraints. Databases Under Evaluation introduces the three platforms, highlighting architectural strengths and justifying selection criteria based on startup accessibility.The Canonical E-commerce Schema specifies the five-table structure and ~787K record dataset representing realistic startup scale, grounding design decisions in established benchmarks, and empirical startup data research.

Data Generation and Loading addresses synthetic dataset creation, documenting how parameterized generation overcomes limitations in existing benchmark datasets while maintaining reproducibility through fixed seeding. Phase-Specific Index Profiles explain the strategic rationale behind read-optimized, write-minimized, and balanced indexing configurations applied sequentially throughout testing.

The core evaluation unfolds in three dedicated sections. Read Phase Strategies and Queries detail the dual-execution protocol (A1 cold/warm analysis, A2 parameter sensitivity) across 10 queries spanning basic to advanced complexity. Write Phase Strategies and Queries describe 8 operation patterns including single-row operations, bulk inserts, conditional updates, and multi-table transactions. Mixed Phase Strategies and Queries explain the 65/35 read-write interleaved workload reflecting production traffic patterns.

Miscellaneous documents log conventions, fairness protocols ensuring unbiased comparison, acknowledged limitations including free-tier constraints and driver-level timing, and replication configurations. Together, these sections establish a comprehensive framework in which each component builds on previous foundations to support rigorous cross-platform performance analysis.

3.1. Objective and Configuration

This study evaluates three cloud-managed database platforms under realistic early-stage e-commerce workloads to address four primary objectives: compare read, write, and mixed throughput performance across document-oriented (MongoDB Atlas), relational (Supabase/PostgreSQL), and wide-column (DataStax Astra DB) architectures; quantify latency distributions (p50, p95, p99) for common e-commerce query patterns including product searches, order history retrieval, and cart operations; assess the performance impact of phase-specific index profiles on read optimization versus write minimization trade-offs; and evaluate database behavior under startup-realistic constraints including free-tier resource limits and single-session execution without concurrent users.

The benchmarking environment employed Python 3.x within Jupyter Notebook for reproducible execution, using database-specific drivers (psycopg2 for PostgreSQL, pymongo for MongoDB, Cassandra-driver for Astra DB) and supporting libraries including pandas for data generation, Faker for synthetic content, numpy for statistical operations, and tqdm for progress monitoring. All tests executed on a Dell G15 5520 laptop (Intel Core i5-12500H, 16GB DDR5 RAM, 512GB SSD, Windows 11) connecting to cloud-hosted database instances. Each default free-tier configuration is maintained on the platform without custom tuning, ensuring comparability and reflecting out-of-box performance accessible to early-stage startups.

3.2. Databases Under Evaluation

This study evaluates three cloud-managed database platforms that represent distinct architectural paradigms while maintaining startup accessibility: MongoDB Atlas (document-oriented NoSQL), Supabase (PostgreSQL-based relational) and DataStax Astra DB (wide-column Cassandra). Each platform provides fully-managed services abstracting infrastructure complexity, enabling production deployment without operational overhead.

MongoDB Atlas delivers flexible schema evolution through document storage, horizontal scaling via automated sharding, and cross-cloud portability across AWS, GCP, and Azure. Supabase extends PostgreSQL's ACID guaranties with integrated authentication, real-time subscriptions, and auto-generated REST APIs, accelerating application development cycles. Astra DB combines Cassandra's distributed architecture with cloud-native automation, providing linear scalability and inherent fault tolerance for applications anticipating rapid growth.

Selection criteria prioritized startup-specific constraints: persistent free tiers eliminating financial barriers (Atlas: 512MB no-limit storage; Supabase: generous APIs quotas; Astra: $ 25 monthly credit for 80GB storage), intuitive web interfaces reducing administrative complexity, streamlined data loading support through visual tools and programmatic APIs, and frictionless registration without credit card requirements. This three-database selection spans primary database paradigms—document, relational, and wide-column—addressing the majority of early-stage application requirements while providing clear migration paths as systems scale.

3.3. Canonical E-commerce Schema

The study employs a five-table relational schema adapted from TPC-W's eight-table e-commerce standard(Menascé, 2002)(Smith, 2000). TPC-W defines minimum structures comprising tables for Customer, Address, Order, Order_Line, Item, Author, Country, and CC_ Xacts tables with established relationships—customers have one-to-many relationships with orders, orders contain multiple order lines, and addresses link to country data (Menascé, 2002). This benchmark architecture scales database size proportionally to workload intensity through fixed multipliers: 2,880 customer records per emulated browser, with dependent tables scaling at defined ratios (addresses at 2×, orders at 0.9×, order lines at 3× orders) (Menascé, 2002)(Smith, 2000). TPC-W specifies five valid scale factors—1K, 10K, 100K, 1M, and 10M items—ensuring comparable results only within identical scales (Menascé, 2002).

This study simplifies to five core tables (Categories, Customers, Products, Orders, Order_Items) while preserving essential e-commerce relationships and scaling principles. The ~787K total record distribution reflects startup-realistic constraints documented in industry research: 65% of early-stage companies manage databases under 100GB, with 76% operating read-heavy or balanced workloads emphasizing short-lived queries (Springtail). The dataset comprises 50 categories, 12,000 products, 25,000 customers, 250,000 orders, and 500,000 order items—proportions aligned with TPC-W's scaling guidance but sized for organizations with

1–50 employees managing single-digit gigabyte databases (Menascé, 2002)(Springtail).

The Real-world design of an e-commerce database goes beyond benchmark minimums to support complete business processes. Research identifies ten components of the logical package, including user accounts with session tracking, dynamic catalog management buffer vendor variations, state modeling of the shopping cart, comprehensive order-invoice-payment workflows, customer service feedback systems and price comparison agents (Song and Whang, 2000). The canonical schema balances normalization with performance through surrogate keys preventing data change cascades, intentional denormalization where business logic demands (statistical attributes in parent tables, foreign key constraints selectively omitted), and weak entity patterns for complete audit trails (Song and Whang, 2000). This approach prioritizes operational simplicity—74% of startups operate single primary databases without complex sharding—while maintaining flexibility for the eight-stage value chain (attract, interact, transact, pay, deliver, personalize, service, customize) critical to e-commerce operations (Springtail).

Premature scaling introduces commitment risk that increases failure rates 20-40% when initiated within six months of founding (Lee and Kim, 2024). Early scalers lacking A/B testing experimentation account for elevated failure odds, as hasty resource commitment curtails learning and traps organizations in poor product-market fit scenarios (Lee and Kim, 2024). The study's startup-scale dataset avoids this trap by aligning size with early-stage operational patterns: preference for managed cloud services (88% adoption), failover-focused replication rather than complex read scaling, and simple application connectivity (53% connect only 1-2 applications) (Springtail). This configuration provides sufficient complexity to test realistic query patterns while maintaining resource constraints characteristic of organizations four years pre-scaling when administrative demands still match founding team capacity (Springtail).

3.4. Data Generation and Loading

Synthetic data generation addresses fundamental benchmark limitations. Research demonstrates that concentrated benchmarking on limited datasets encourages overfitting through hyperparameter optimization targeting test artifacts rather than generalizable patterns (Longjohn, Kelly, Singh and Smyth, 2024). Documented failures include Iris dataset version inconsistencies producing incomparable results, BUPA's misidentified class labels generating meaningless outcomes, and Statlog's documentation errors persisting 25 years across thousands of citations (Blagec, Kraiger, Frühwirt and Samwald, 2023). TPC-W's Customer Behavior Model Graph provides only partial user behavior representation, failing to model dynamic role transitions between browsing and purchasing within single sessions (Pena-Ortiz, Gil, Sahuquillo and Pont, 2012). The benchmark generates fixed patterns reflecting Web 1.0 passive consumers rather than Web 2.0's participative contributors, with workloads differing from production systems by orders of magnitude—TPC-C eliminates idle time while real systems exhibit diurnal variation with distinct load concentrations (Kavalanekar, Narayanan, Sankar, Thereska, Vaid and Worthington, 2009).

Existing e-commerce benchmarks focus on basic intents (product finding, purchasing) while real users pursue complex goals requiring multi-step reasoning: voucher optimization, budget satisfaction, shipping qualification (Wang, Xiao, Sun, Zhao, Luo, Zhang and Zeng, 2025). This study's synthetic framework employs fixed seed (42) for reproducibility while implementing realistic distributions—50 categories reflecting taxonomy depth, customer-to-order ratios modeling repeat purchases (25K customers generating 250K orders), and order-to-item distributions capturing basket sizes (500K items averaging 2 per transaction)—capturing workload features absent from fixed TPC-W datasets (Wang et al., 2025).

Data loading employed database-native approaches: Supabase utilized Python-driven CSV generation with pgAdmin bulk import; MongoDB leveraged Compass GUI for direct JSON import; AstraDB required hybrid methodology with cassandra-driver achieving 500–600 rows/second via execute_concurrent_with_args. All databases loaded ~787K records sequentially (Categories → Customers → Products → Orders → Order_Items) within 5 minutes for relational systems, 25–30 minutes for distributed AstraDB. Critical challenges included PostgreSQL nullable integer type errors requiring pandas Int64 dtype, TEXT[] array formatting requiring custom escaping, JSON serialization via json.dumps(), and post-import sequence synchronization. AstraDB omitted SET<TEXT> tags due to Windows CMD limitations and employed dynamic schema introspection for cross-phase portability. Verification occurred through UI-based row counts and CQL commands confirming dataset integrity before benchmark execution.

3.5. Phase-Specific Index Profiles

Database indexes accelerate read operations through efficient data pathways but impose maintenance overhead during writes—a trade-off critical for resource-constrained startups. This study employs phase-specific index profiles aligning with distinct workload characteristics rather than uniform indexing strategies.

The read-optimized phase applies comprehensive indexing supporting common e-commerce patterns: composite indexes (category_id, price DESC) for sorted category browsing and (customer_id, order_date DESC) for recent order history; single-column indexes on popularity for top product lists, name_lc for case-insensitive searches; unique constraints on sku_lc and email_lc for data integrity. This configuration tests maximum query performance when systems prioritize retrieval operations.

The write-minimized phase retains only business-critical uniqueness constraints (email_lc, sku_lc) and minimal order_items.order_id index for order assembly, eliminating all read-optimization composites. This reduction isolates pure

write performance without index maintenance amplification during insert, update, and delete operations.

The balanced phase restores key composite indexes—(category_id, price DESC) and (customer_id, order_date DESC)—plus lightweight single-column indexes (popularity DESC, name_lc) supporting the 65/35 read-write workload mix. This configuration maintains common read path efficiency while limiting write overhead to truly useful indexes, reflecting realistic startup operational patterns where applications simultaneously serve browsing traffic and transaction processing.

3.6. Read Phase Methodology

The read query selection reflects realistic e-commerce user interactions documented in established benchmarks and behavioral research. TPC-W defines 14 web interactions categorized into browse operations (product detail, search request, best sellers, new products) and order operations (shopping cart, buy request, order inquiry), representing the fundamental navigation patterns in online commerce (Khouja, Kamoun, Lladó and Puigjaner, 2010)(Dodge, Menascé and Barbára, 2001)(Menascé, 2002). Browse interactions constitute 80-95% of e-commerce traffic depending on user intent, while order interactions comprise the remaining 5-20% (Khouja et al., 2010).

This study implements 10 read queries spanning three complexity tiers to capture the full spectrum of e-commerce browsing behavior. Basic queries (product detail lookup, customer order history, category browse) map directly to TPC-W's most frequent interactions—Product Detail (12.35-21% of traffic), Order Display, and Home interactions respectively (Menascé, 2002)(Khouja et al., 2010). These operations represent single-table or simple join patterns with minimal processing requirements. Moderate queries (search with filters, best sellers, new arrivals, order detail with items) correspond to TPC-W's Search Request (14.53-20% of traffic), Best Sellers (0.46-11% depending on mix), and New Products interactions, requiring multi-table joins and sorted result sets (Khouja et al., 2010)(Menascé, 2002). Advanced queries (facet counts under filters, trending products, JSON attribute filtering) address complex analytical patterns involving window functions, nested aggregations, and multi-dimensional filtering.

Faceted search operations (R4, R7) merit particular attention as they represent the dominant navigation pattern in modern e-commerce (Petzka, Stadler, Katsimpras, Haarmann and Lehmann, 2017). Faceted browsing enables users to apply filter restrictions across multiple dimensions—category, price, brand, attributes—to progressively narrow search spaces (Petzka et al., 2017). Research demonstrates that facet counting (displaying result counts for each filter value before selection) is critical for preventing empty result sets and guiding users toward productive searches (Petzka et al., 2017). Product search has evolved to become the primary driver of e-commerce traffic, with retailer site searches increasing 300% from 2010-2012 and reaching 634 million monthly searches compared to just 134 million at traditional comparison sites (Baye, De los Santos and Wildenbeest, 2012).

User behavior studies confirm that browsing, searching, and purchasing constitute three fundamental interaction categories (Anonymous, 2023). Browsing behavior encompasses catalog navigation and popular page identification; searching includes product lookup and comparison activities using both attribute-based filtering (comparing prices across products) and alternative evaluation (examining multiple features of a single product) (Nah, Hong, Chen and Lee, 2012); purchasing behavior covers the cart-to-checkout conversion path (Anonymous, 2023). Under time pressure, users view fewer products (23.6 vs 33.967 products) but maintain consistent search intensity, demonstrating that search and lookup operations remain essential regardless of session constraints (Shih, Lu and Shih, 2019).

The dual-execution protocol tests both cache effects and parameter sensitivity. Phase A1 executes 1 cold run followed by 10 warm runs per query using fixed parameters, measuring performance improvements from database caching and query plan reuse. Phase A2 executes 10 runs per query with randomized parameters drawn from the dataset, testing latency stability across different input distributions and cardinalities. This approach isolates workload-independent performance (A1 warm runs) from parameter-dependent variance (A2), providing startup operators insight into both best-case cached performance and realistic variable-input behavior.

3.7. Write Phase Methodology

The write operation selection reflects e-commerce transactional requirements documented in OLTP benchmarks and ACID compliance literature. TPC-W categorizes web interactions into browse and order types, with order interactions characterized by greater processing requirements including database writes, secure SSL processing, and ACID transaction enforcement (Smith, 2000). Order pages—Shopping Cart, Customer Registration, Buy Request, and Buy Confirm—constitute 5-20% of traffic depending on workload mix but demand substantially higher database resources than browse operations (Smith, 2000). The benchmark mandates that databases support lookup, insert, and update functionality with explicit commit and rollback capabilities across minimum eight interconnected tables (Menascé, 2002).

This study implements 8 write operation types encompassing single-row, bulk, conditional, and transactional patterns critical for e-commerce systems. Single-row operations (customer registration, cart item addition, profile updates) map to TPC-W's Registration and Shopping Cart interactions, requiring INSERT and UPDATE operations with secure authentication (Smith, 2000). Bulk inserts (batch order_items loading) reflect real-world data migration and order processing scenarios where batching provides 98% execution time reduction compared to sequential single-row operations—1,000 operations requiring 2,615ms via table-valued parameters versus 128,852ms for sequential inserts (Microsoft Corporation, 2023). Conditional updates

and deletes test filtered operations on customer and product tables. Transactional write patterns (order placement with multi-table inserts) mirror TPC-C's NewOrder transaction, which performs 2+I reads, 1+I updates, and 2+I inserts across seven tables to atomically create orders, update inventory, and record line items (Tözün, Pandis, Kaynak, Jevdjic and Ailamaki, 2013).

ACID property enforcement is central to e-commerce write benchmarking (Tözün et al., 2013). Atomicity ensures all-or-nothing execution through transaction commit and rollback mechanisms—TPC-C's NewOrder includes 1% intentional failure rate requiring rollback, while real implementations show 18,372 transaction retries across 142,620 executions (Panpaliya, 2020). Consistency is validated through four post-execution tests verifying warehouse-district balance equality, order ID sequence integrity, new order count accuracy, and order line count correctness (Tözün et al., 2013). Isolation is achieved through snapshot reads and multi-version concurrency control, preventing concurrent transactions from interfering while maintaining throughput (Panpaliya, 2020). Durability is guaranteed through replication mechanisms where write operations propagate to majority nodes before acknowledgment, with MongoDB benchmarks demonstrating that snapshot reads with majority writes provide highest consistency without compromising performance (Panpaliya, 2020).

The TPC-C transaction mix validates write operation distribution (Tözün et al., 2013). NewOrder (45%) represents mid-weight read-write transactions introducing variable load based on item count; Payment (43%) comprises light-weight updates to customer balance and district statistics; Delivery (4%) executes deferred batch processing of up to 10 orders via queuing; Order-Status and Stock-Level (4% each) provide read-only operations for workload diversity (Tözün et al., 2013). This 92% read-write, 8% read-only distribution stresses transaction processing engines—particularly logging and concurrency control modules—reflecting production e-commerce intensity where order placement and payment processing dominate activity (Difallah, Pavlo, Curino and Cudre-Mauroux, 2013).

Batch processing significance is empirically demonstrated through performance testing showing dramatic throughput improvements (Microsoft Corporation, 2023). Wrapping 1,000 sequential operations in transactions reduces execution time from 21,479ms to 2,756ms (87% improvement) within same datacenter due to delayed transaction log flushing (Microsoft Corporation, 2023). Table-valued parameters achieve 98.2% reduction (21,479ms to 382ms) by minimizing network overhead—fixed costs of serialization, transfer, and deserialization are amortized across batch rather than repeated per operation (Microsoft Corporation, 2023). SqlBulkCopy provides optimal performance for batches exceeding 1,000 rows, demonstrating 12-31% faster execution than table-valued parameters at scale (Microsoft Corporation, 2023). Tests confirm single large batches outperform subdivided batches (347ms for 1,000-row batch versus 630ms for 20×50-row batches), and parallel execution provides no benefit due to contention overhead (Microsoft Corporation, 2023).

Each write operation type executes approximately 100 times for statistical confidence, with driver-level timing capturing execution latency including network transfer, transaction processing, and acknowledgment receipt. Error handling logs all attempts including constraint violations and timeout scenarios without halting execution, providing realistic failure rate measurements under free-tier resource constraints.

3.8. Mixed Throughput Phase Methodology

The mixed throughput phase evaluates database performance under realistic e-commerce traffic patterns combining concurrent read and write operations. TPC-W defines three web interaction mixes representing different user behavior profiles: browsing mix (95% browse, 5% order), shopping mix (80% browse, 20% order), and ordering mix (50% browse, 50% order) (Menascé, 2002)(Smith, 2000). The browsing mix simulates window-shopping behavior where users primarily explore products with minimal purchasing, placing pressure on web servers and caching layers (Menascé, 2002). The shopping mix represents balanced e-commerce sessions with moderate conversion rates (Smith, 2000). The ordering mix models purchase-focused sessions where users complete transactions efficiently, stressing database servers through high-volume transactional writes (Menascé, 2002). These mixes reflect real user behavior statistics, with different customer patterns generating distinct loads on system resources (Menascé, 2002).

This study implements a 65% read, 35% write mixed workload balancing browsing intensity with transactional activity typical of startup e-commerce platforms. The read distribution emphasizes high-frequency operations: product lookup (20%), search with filters (22%), top products (8%), order history (7%), recent orders (4%), and customer profile (4%). The write distribution prioritizes cart operations (add to cart 12%, update cart 8%) and order completion (place order transaction 10%), with lower-frequency operations including registration (2%) and profile updates (3%). Operations execute in randomized sequence for a fixed total count (500 operations), simulating organic user navigation rather than artificial patterns.

Mixed workload benchmarking addresses fundamental limitations of isolated read-only and write-only testing (Kaszó and Levendovszky). YCSB demonstrates that read-write ratio significantly impacts throughput and latency—MongoDB achieves 50% higher throughput under 90% read workloads compared to 90% write workloads, while Cassandra exhibits 30% throughput variation with peak performance degradation at balanced 50/50 distributions ten. Each database responds differently to ratio variations, with performance differences reaching 50% at identical load levels (Kaszó and Levendovszky). Isolated testing fails to capture disk and cache contention effects

that emerge when operations execute concurrently—OLTP-Bench experiments revealed that ReadModifyWrite transactions achieved higher throughput than UpdateRecord operations specifically due to preceding DeleteRecord phase interactions, an anomaly impossible to detect through separate measurement (Difallah et al., 2013).

The rationale for predefined operation mixes rather than isolated tests centers on real-world system behavior representation ten. Cloud serving benchmarks explicitly avoid modeling single applications, instead examining wide workload characteristic ranges to identify performance boundaries (Kaszó and Levendovszky). YCSB's workload profiles model actual applications: session stores (50% read, 50% update), photo tagging (95% read, 5% update), and user status feeds (95% read, 5% insert), each reflecting documented production traffic patterns ten. OLTP-Bench provides empirical evidence that mixed workloads enable detection of performance anomalies—MySQL throughput degradation after 2600 seconds under sustained YCSB load revealed buffer pool exhaustion requiring page eviction, a production-critical issue never discoverable through isolated testing (Difallah et al., 2013).

Hybrid OLTP-OLAP benchmarking validates concurrent mixed workload necessity for modern systems (Funke, Kemper and Neumann, 2011)(Cole, Funke, Giakoumakis, Guy, Kemper, Krompass et al., 2011). TPC-CH experiments demonstrate that increasing OLAP sessions from 3 to 12 enhances query throughput from 0.38 to 1.20 queries per second but causes query execution times to increase 20-30% while OLTP throughput declines 14%, quantifying resource contention between concurrent transactional and analytical operations (Funke et al., 2011). System comparisons reveal that OLAP query performance cannot be inferred from isolated measurements—Query Q18 response time increased from 30,271ms (pure OLAP) to 96,159ms (8 concurrent OLTP streams), demonstrating 3x degradation under realistic mixed loads (Cole et al., 2011). The CH-benCHmark metric comparing isolated versus mixed execution (13.3 isolated versus 20.0 mixed) proves that concurrent workload performance cannot be predicted from separate measurements, with OLAP queries suffering disproportionately from parallel execution (Cole et al., 2011).

This mixed phase executes operations sequentially within a single session with driver-level timing capturing end-to-end latency including network transfer, transaction processing, and acknowledgment. The randomized operation sequence with realistic ratio distribution provides startup operators comprehensive understanding of database behavior under production-representative e-commerce traffic patterns combining product discovery, cart management, and order completion activities.

3.9. Miscellaneous

Logging Strategy: All phases employed standardized CSV logging capturing universal metadata (timestamp_utc, run_id, db_system, server_version, driver version, connection_params, db_name) ensuring traceability across engines. Read phase generated reads_A1.csv (11 runs per query: 1 cold + 10 warm with fixed parameters) and reads_A2.csv (10 runs with randomly generated parameters). Write phase produced writes.csv logging approximately 100 iterations per operation pattern. Mixed phase created throughput.csv recording 500 operations (65% read, 35% write) with sequence_number tracking execution order plus summary metrics (QPS, TPS). Each row captured query_id, complexity classification, execution_ms, row_count or rows_affected, error_code, error_message, and params_json, with failures logged without halting execution.

Fairness and Parity: Comparable-by-construction methodology ensured identical synthetic dataset (seed 42) loaded across all databases, functionally equivalent queries using native syntax, standardized execution protocols (A1/A2 runs, write iterations, mixed ratios), phase-specific indexing applied consistently, driver-level timing with uniform logging schema, single hardware environment, and maintained free-tier constraints without premium optimizations.

Limitations: Evaluation reflects free-tier resource constraints, single geographic region deployments (nearest available datacenters), single-session execution without concurrent users, and driver-level timing capturing network transfer overhead rather than isolated server-side processing. Results represent out-of-box performance accessible to early-stage startups rather than enterprise-tuned configurations.

Replication: All databases operated under default free-tier replication settings without custom high-availability configurations.

4. Results

This section presents empirical findings from a comprehensive e-commerce database benchmarking study comparing Supabase/PostgreSQL, MongoDB Atlas, and DataStax AstraDB across four distinct workload phases. Each database executed identical query sets under controlled conditions: Phase A1 (fixed-input reads with read-optimized indexes), Phase A2 (random-input reads with read-optimized indexes), Write Phase (isolated write operations with write-minimized indexes), and Mixed Phase (65% read, 35% write workload with balanced indexes). All performance data was captured in structured CSV logs during execution and subsequently processed through automated metric computation scripts to derive percentile distributions (p50, p95, p99), dispersion indicators (IQR, MAD), throughput metrics (QPS, TPS), and reliability measures (success rates). The analysis pipeline generated 10 figures visualizing cross-database comparisons, with outlier operations isolated for clarity where performance deviations exceeded 10× median thresholds. Observations reported here are factual measurements; interpretation and implications are reserved for subsequent discussion.

4.1. Results Framework and Analysis Strategy

4.1.1. Metric Selection Rationale

Performance evaluation employed percentile-based distributions rather than arithmetic means to capture tail behaviour critical for user-facing applications. The 50th percentile (p50 or median) represents typical latency experienced by half of all requests, while the 95th (p95) and 99th (p99) percentiles quantify worst-case performance affecting minority but significant user populations. Unlike averages, percentiles remain robust against outliers and directly answer operational questions: "What latency do 99% of users experience?" Maximum values (max) captured absolute worst-case scenarios. This approach aligns with industry-standard service-level objective (SLO) definitions where tail latencies often dominate perceived application responsiveness.

Phase-specific metrics extended this foundation. For A1 (fixed-input reads), cold-warm delta $\Delta_{\text{cold-warm}}$ and percentage improvement % improvement quantified cache effectiveness. Tail amplification ratio $\frac{\text{p99}}{\text{p50}}$ measured consistency—values below 2 indicate stable performance, while ratios exceeding 5 signal unpredictable tail latency unsuitable for latency-sensitive applications. For A2 (random-input reads), dispersion indicators complemented percentiles: Interquartile Range

$$\text{IQR} = Q3 - Q1$$

captured middle 50% spread, while Median Absolute Deviation

$$\text{MAD} = \text{median}\left(|\text{value} - \text{median}|\right)$$

provided outlier-resistant variability measures. These metrics directly quantified parameter sensitivity under realistic access patterns. Write phase metrics included success rates

$$\text{Success rate} = \frac{\text{successes}}{\text{total}} \times 100$$

and error taxonomy to assess reliability alongside latency. Mixed phase introduced throughput metrics: Read QPS (queries per second)

$$\text{Read QPS} = \frac{\text{total reads}}{\text{elapsed time}}$$

Write TPS (transactions per second)

$$\text{Write TPS} = \frac{\text{total writes}}{\text{elapsed time}}$$

enabling realistic 65/35 workload assessment under balanced indexes.

4.1.2. Data Processing Pipeline

Raw execution logs captured via the logging strategy outlined in Methodology (Section 3.9) were processed through four phase-specific Python scripts following a standardized eight-cell architecture: (1) dependency imports, (2) manual path configuration, (3) CSV ingestion with column validation, (4) sample count and data integrity verification, (5) percentile computation using NumPy linear interpolation, (6) phase-specific metric derivation, (7) output aggregation to structured CSVs, and (8) validation summary generation. All scripts employed consistent column naming (db_system, query_id, execution_ms, error_code) ensuring database-agnostic processing. Error handling excluded failed operations (missing execution_ms or non-null error_code) from latency calculations while preserving error metadata for reliability analysis. Run count validation flagged anomalies—A1 expected exactly 1 cold + 10 warm runs per query, Write and A2 expected approximately 100 runs each, Mixed validated 65±5% read ratio. Percentile functions gracefully handled edge cases (empty series, insufficient samples) by returning None rather than crashing, maintaining pipeline robustness across diverse database behaviors. All outputs embedded database identifiers for traceability, enabling reproducible metric recomputation from archived logs.

4.1.3. Visualization Strategy

Ten publication-quality figures (300 DPI PNG) were generated via automated scripts to enable cross-database comparison while managing scale compression from performance outliers. Outlier detection applied dual thresholds: operations flagged if maximum latency exceeded 10× overall median p50 OR 5× overall p99. Detected outliers (e.g., AstraDB Query R6 at 413 seconds) were isolated into separate figures, allowing main visualizations to use adaptive Y-axis scaling optimized for normal query ranges—20ms padding for ranges <100ms, 15% proportional padding otherwise. This dual-output approach preserved both typical performance clarity and outlier context without scale distortion. Color semantics remained consistent: green for normal/successful operations, orange for moderate severity (writes, tail latencies), red for errors/outliers/high amplification ratios, and purple for distribution visualizations (A2 violins). A1 generated 3 figures (warm performance bars, tail amplification heatmap, outliers if detected), A2 produced 2 figures (IQR comparison bars with optional outlier separation), Write phase created 2 figures (dual-panel latency + success rates, outlier latency if detected), and Mixed phase yielded 3 figures (throughput comparison, per-operation p99 bars, outlier operations if detected). Figure types prioritized quantitative comparison over qualitative distribution visualization—grouped bar charts dominated over violin plots—to maximize cross-database decision-making utility. All figures included stats boxes documenting query/operation counts, outlier presence, and metric definitions, ensuring interpretability without external reference.

4.2. Phase A1: Fixed-Input Read Performance

Phase A1 evaluated read query performance under fixed-parameter conditions with read-optimized indexes across ten queries (R1–R10) spanning basic, moderate, and advanced complexity levels. Each query executed once cold followed by ten warm iterations to assess cache effectiveness. Table 2

presents warm-run latency distributions across all databases, while Table 3 reports tail amplification ratios and cold-to-warm performance improvements.

Warm-Run Latency: MongoDB Atlas demonstrated the lowest median warm p50 at 42.72 ms and median p99 at 43.84 ms, maintaining consistent sub-50ms response times across all queries except R9 (49.79 ms) and R10 (45.61 ms). Supabase/PostgreSQL exhibited median warm p50 of 96.00 ms and median p99 of 97.20 ms, with highest latencies observed in R5 (136.03 ms p99) and R9 (133.97 ms p99). DataStax AstraDB recorded median warm p50 of 202.65 ms and median p99 of 235.71 ms under normal conditions, but queries R6 and R10 triggered extreme outlier behavior with p99 values of 413,818.83 ms and 2,349.88 ms respectively, exceeding thresholds by 850× and 24× the overall median p50. These outliers, isolated in Figure 2, corresponded to best-sellers aggregation (R6) and attribute-based filtering with client-side sorting (R10). The remaining eight queries displayed stable performance within expected Cassandra latency ranges, as shown in Figure 1.

Tail Behavior Consistency: Tail amplification ratios (p99/p50) quantified performance predictability. MongoDB maintained the tightest consistency with median amplification of 1.02, indicating p99 latencies remained within 2% of median values for most queries, except R9 (1.161) and R10 (1.076). Supabase achieved median amplification of 1.01, demonstrating exceptional stability with all queries below 1.04 except R1 (1.038) and R8 (1.018). AstraDB exhibited median amplification of 1.13 for normal queries, but R10's extreme tail produced amplification of 4.42, signaling high unpredictability under random parameter variation. Figure 3 visualizes these ratios via heatmap, with green cells (<2.0 ratio) dominating Supabase and MongoDB rows, while AstraDB displayed orange-to-red cells for R5 (1.261), R9 (1.160), and R10 (4.422).

Cache Effectiveness: Cold-to-warm percentage improvements revealed variable cache utilization. Supabase demonstrated dramatic improvement on R5 (97.68% reduction from 5,785.13 ms cold to 134.32 ms warm), indicating effective query result caching for aggregation-heavy operations. MongoDB showed minimal cold-warm deltas (median 0.43% improvement), suggesting warm cache already active during cold runs or negligible cold-start overhead. AstraDB achieved substantial improvements on R1 (22.79%), R7 (11.01%), R9 (18.08%), and R5 (11.53%), but recorded negative improvements on R3 (−1.95%) and R10 (−28.98%), indicating cold executions occasionally outperformed warm runs—likely attributable to partition-level cache eviction or coordinator node selection variability in distributed Cassandra architecture.

4.3. Phase A2: Random-Input Read Performance

Phase A2 assessed read query performance under random parameter variation with read-optimized indexes maintained from A1. Each query executed ten times with randomized input parameters (product IDs, customer IDs, date ranges, category filters) to simulate realistic production access patterns. Table 4 presents latency distributions and IQR dispersion measures, while Table 5 reports MAD values and tail amplification ratios under parameter sensitivity stress.

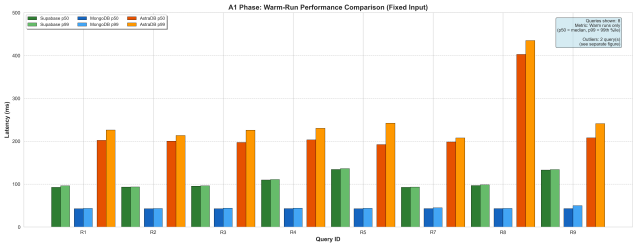

Figure 1: Phase A1 Warm-Run Latency Distributions Across Databases

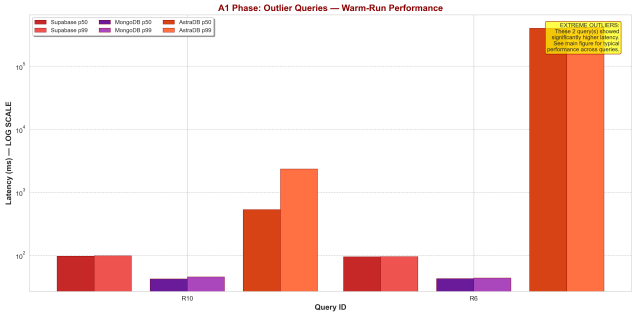

Figure 2: Phase A1 Outlier Query Performance (R6, R10)

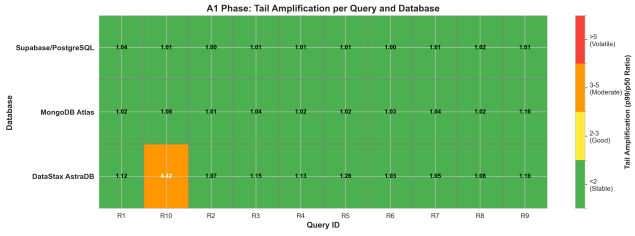

Figure 3: Phase A1 Tail Amplification Ratio Heatmap

Random-Input Latency: MongoDB Atlas sustained median p50 of 42.78 ms and median p99 of 46.35 ms, demonstrating minimal degradation from A1 fixed-input conditions (42.72 ms and 43.84 ms respectively). Supabase/PostgreSQL recorded median p50 of 95.78 ms and median p99 of 98.65 ms, remaining consistent with A1 warm performance (96.00 ms and 97.20 ms). DataStax AstraDB exhibited median p50 of 198.87 ms and median p99 of 211.03 ms for normal queries, but R6 and R10 again triggered extreme outliers with p99 values of 388,468.93 ms (386 seconds) and 1,469.58 ms (1.5 seconds) respectively. Query R6 deteriorated further from A1's 413 seconds to 386 seconds median p50, indicating consistent aggregation performance issues. Query R8 (order detail with items) displayed elevated baseline latency at 384.71 ms p50, suggesting JOIN-equivalent operation overhead in Cassandra's denormalized model. Figure 4 visualizes IQR distributions across databases, with R6 and R10 isolated in Figure 5 due

Table 2
Phase A1 Warm-Run Latency (ms)

Q	Supa		Mongo		Astra	
	p50	p99	p50	p99	p50	p99
R1	92.5	96.0	42.7	43.5	202	226
R2	93.0	93.4	42.5	43.0	200	214
R3	95.3	96.2	42.5	44.0	197	226
R4	110	111	43.0	43.8	203	230
R5	134	136	42.6	43.7	192	242
R6	95.3	95.7	42.8	43.9	400k	414k
R7	92.6	93.2	42.9	44.8	198	208
R8	96.7	98.5	42.8	43.4	403	435
R9	133	134	42.9	49.8	208	241
R10	97.1	98.3	42.4	45.6	531	2350

Table 3
Phase A1 Tail Amp & Cache

Q	Amp			% Impr		
	S	M	A	S	M	A
R1	1.04	1.02	1.12	19.3	1.2	22.8
R2	1.01	1.01	1.07	13.2	0.4	−0.2
R3	1.01	1.04	1.15	−0.1	−1.1	−2.0
R4	1.01	1.02	1.13	7.4	−1.7	6.6
R5	1.01	1.02	1.26	97.7	−0.1	11.5
R6	1.00	1.03	1.03	−2.7	−1.1	3.0
R7	1.01	1.05	1.05	2.3	1.6	11.0
R8	1.02	1.02	1.08	6.7	−0.2	0.3
R9	1.01	1.16	1.16	0.2	−0.3	18.1
R10	1.01	1.08	4.42	−4.8	3.9	−29

to IQR values of 1,246.54 ms and 866.32 ms respectively—exceeding the 13.51 ms threshold by 92× and 64×.

Parameter Sensitivity: IQR measurements quantified variability induced by random parameter draws. MongoDB achieved tightest parameter control with median IQR of 0.84 ms, indicating middle 50% of observations confined to sub-millisecond range. Supabase recorded median IQR of 0.80 ms, outperforming MongoDB marginally in dispersion control. AstraDB exhibited median IQR of 6.63 ms for normal queries (7.9× higher than MongoDB), reflecting partition key distribution sensitivity—queries targeting hot partitions experienced lower latency while cold partition reads incurred coordinator-to-node routing overhead. MAD analysis corroborated IQR findings: Supabase median MAD of 0.47 ms, MongoDB 0.40 ms, AstraDB 2.89 ms. Queries R1 and R5 demonstrated exceptional parameter insensitivity across all databases (IQR < 2 ms), while R4 (search with facets) and R10 (attribute filtering) exhibited higher sensitivity, particularly in MongoDB (R10 IQR 5.21 ms, 6.2× database median) and AstraDB (R10 IQR 866.32 ms excluding R6 outlier).

Tail Consistency Under Randomization: Tail amplification ratios remained stable relative to A1 despite parameter variation. MongoDB maintained median amplification of 1.05, though R10 escalated to 1.692 (69% p99 degradation from p50) due to client-side JSON attribute filtering overhead. Supabase achieved median amplification of 1.01, with R4 exhibiting highest amplification at 1.056. AstraDB recorded median amplification of 1.04 for normal queries, but R10's extreme tail produced amplification of 2.585, indicating 2.6× p99 inflation relative to median—a significant degradation from A1's 4.422 ratio, suggesting partial mitigation via randomized partition access reducing hot-spot contention. Queries R5, R6, R7, and R8 maintained amplification below 1.07 across all databases, demonstrating tail stability for aggregation and JOIN-simulation operations when parameters vary within expected ranges.

4.4. Phase Write: Isolated Write Performance

Write phase evaluated isolated write operation performance under write-minimized index configurations across eleven operation types (W1A–W7B) spanning basic single-row mutations, moderate bulk operations, and advanced transactional workflows. Each operation executed 100 times

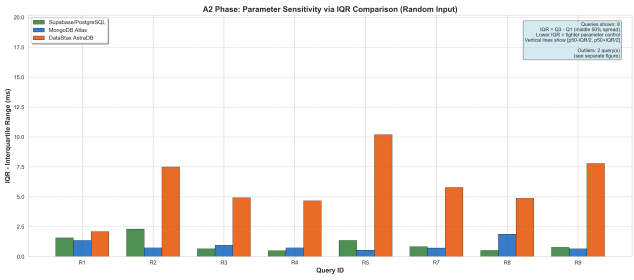

Figure 4: Phase A2 Interquartile Range (IQR) Dispersion Comparison

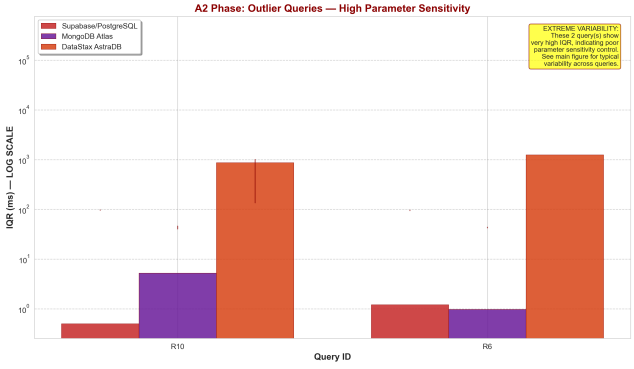

Figure 5: Phase A2 Outlier Query IQR Distribution (R6, R10)

to assess latency distributions and reliability under controlled write-only conditions. Table 6 presents write latency comparisons, while Table 7 reports success rates and error counts. Supabase lacked W6 (transaction rollback test), and AstraDB omitted W3A (conditional update) and W6 due to architectural constraints.

Write Latency Performance: MongoDB Atlas achieved lowest median write p50 at 55.12 ms and median p99 at 103.91 ms, demonstrating superior write throughput across most operation types. Single-row operations (W1B, W1C, W4) completed within 42–55 ms p50 and 52–78 ms p99. AstraDB recorded median p50 of 192.53 ms and median p99 of 212.31 ms, maintaining consistent 190–205 ms latency for basic operations (W1A–W1C, W2, W4, W5) via eventually-consistent writes. Supabase/PostgreSQL exhibited median p50 of 258.21 ms and median p99

Table 4
Phase A2 Random-Input Latency and Dispersion (ms)

Q	Supa			Mongo			Astra		
	p50	p99	IQR	p50	p99	IQR	p50	p99	IQR
R1	93.8	95.2	1.6	42.7	45.9	1.4	190	195	2.1
R2	95.8	99.8	2.3	42.3	43.1	0.7	187	204	7.5
R3	95.2	96.7	0.6	42.5	46.9	1.0	201	217	4.9
R4	109	115	0.5	43.1	51.0	0.7	197	206	4.7
R5	133	134	1.4	42.8	45.1	0.5	189	202	10.2
R6	95.2	97.9	1.2	43.0	49.6	1.0	386k	388k	1247
R7	92.5	93.5	0.8	42.4	45.5	0.7	193	201	5.8
R8	95.7	96.2	0.5	42.2	44.4	1.9	385	399	4.9
R9	132	134	0.8	43.2	53.4	0.7	209	216	7.8
R10	96.3	99.4	0.5	43.3	73.2	5.2	569	1470	866

Table 5
Phase A2 Parameter Sensitivity (MAD & Amp)

Q	Supa		Mongo		Astra	
	MAD	Amp	MAD	Amp	MAD	Amp
R1	0.81	1.02	0.75	1.07	1.13	1.03
R2	1.33	1.04	0.34	1.02	2.19	1.09
R3	0.37	1.02	0.32	1.10	2.29	1.08
R4	0.28	1.06	0.40	1.18	2.77	1.05
R5	0.72	1.01	0.32	1.05	4.60	1.07
R6	0.71	1.03	0.60	1.15	736	1.01
R7	0.46	1.01	0.22	1.07	1.70	1.04
R8	0.31	1.01	1.08	1.05	3.00	1.04
R9	0.48	1.01	0.40	1.24	4.66	1.04
R10	0.29	1.03	0.71	1.69	369	2.59

of 479.16 ms, with single-row mutations (W1A, W1B, W4, W7A, W7B) clustered around 255–258 ms p50, but W1C (single-row delete) and W7A/W7B (indexed column updates) triggered tail latency spikes reaching 510–606 ms p99, suggesting occasional write-ahead log (WAL) flush delays or constraint validation overhead. Bulk operations (W2) and transactional workflows (W5) introduced higher baseline latency: Supabase 430–513 ms p50, MongoDB 38–440 ms p50, AstraDB 190–192 ms p50.

Conditional Operations and Outliers: Conditional write operations (W3A, W3B) requiring predicate evaluation exhibited elevated latency across all databases. W3B (conditional delete with zero-quantity cleanup) triggered the sole Write phase outlier with AstraDB p99 at 1,979.70 ms (1.98 seconds), exceeding the 1,315.63 ms threshold. MongoDB W3B recorded p99 of 1,522.99 ms, while Supabase maintained 554.50 ms p99. Conditional update (W3A) showed similar patterns where executed: Supabase 258 ms p50/283 ms p99, MongoDB 380 ms p50/595 ms p99. The W3B outlier, isolated in Figure 7, reflects full-table scan requirements for condition matching in Cassandra's partition-key-centric model—predicates not aligned with partition keys necessitated coordinator-to-replica broadcast queries across all nodes. MongoDB's conditional operations leveraged index-supported query planning, while Supabase employed bitmap heap scans with sequential predicate evaluation. Figure 6 visualizes latency distributions for normal operations and 100% success rates across all executed operations.

Write Reliability and Index Maintenance: All three databases achieved 100% success rates (Table 7), with zero errors recorded across 3,000 total write executions (11 operations × 100 runs × 3 databases, accounting for missing operations). MongoDB W6 (transaction rollback on duplicate key constraint) completed successfully at 81.18 ms p50/103.91 ms p99, demonstrating robust ACID transaction handling. Index-aware operations (W7A vs. W7B) revealed differential overhead: Supabase showed minimal distinction (255.62 ms vs. 255.83 ms p50), but tail latencies diverged (510 ms vs. 532 ms p99), suggesting index update batching variability. MongoDB exhibited marginal overhead (54.96 ms vs. 55.12 ms p50), while AstraDB maintained near-identical performance (197.61 ms vs. 198.84 ms p50), consistent with Cassandra's lightweight secondary index implementation via hidden materialized views.

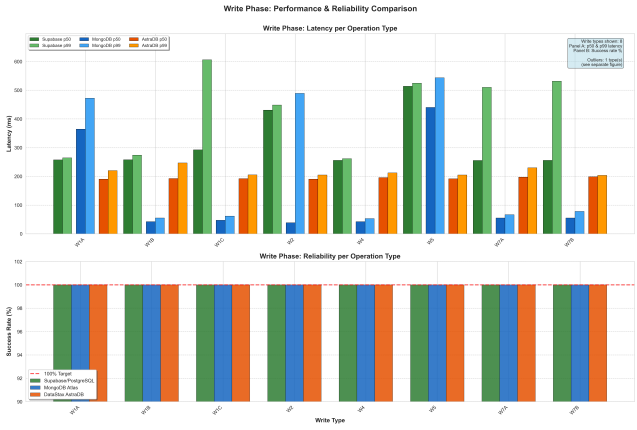

Figure 6: Phase Write: Latency and Success Rate Comparison (Dual Panel)

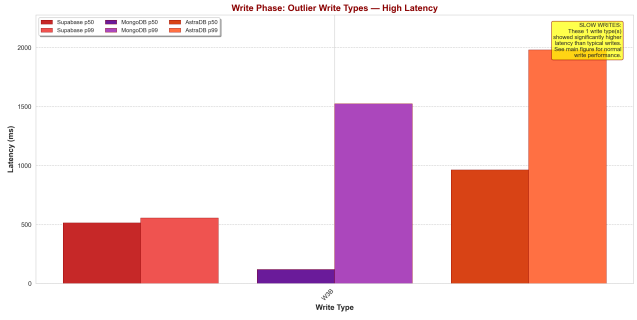

Figure 7: Phase Write: Outlier Operation Performance (W3B)

4.5. Phase Mixed: Realistic 65/35 Workload

Mixed phase evaluated concurrent read-write performance under balanced indexing (neither read-optimized nor write-minimized) with a realistic 65% read and 35% write operation distribution. The workload executed 500 total operations (325 reads across six query types R1–R6, 175 writes across five operation types W1–W5) in randomized interleaved fashion to simulate production e-commerce traffic. Table 8 presents aggregate throughput metrics, while

Table 6
Phase Write Latency Comparison (ms)

Write	Supa		Mongo		Astra	
	p50	p99	p50	p99	p50	p99
W1A	257	264	364	472	191	220
W1B	258	274	42.2	55.2	193	247
W1C	293	606	47.3	61.8	192	205
W2	431	448	38.9	489	190	205
W3A	258	283	380	595	—	—
W3B	513	555	120	1523	962	1980
W4	256	262	42.5	53.0	196	212
W5	514	524	440	543	192	205
W6	—	—	81.2	104	—	—
W7A	256	510	55.0	66.5	198	230
W7B	256	532	55.1	78.3	199	204

Table 7
Phase Write Reliability Assessment

Write	Supa		Mongo		Astra	
	%	Err	%	Err	%	Err
W1A	100	0	100	0	100	0
W1B	100	0	100	0	100	0
W1C	100	0	100	0	100	0
W2	100	0	100	0	100	0
W3A	100	0	100	0	—	—
W3B	100	0	100	0	100	0
W4	100	0	100	0	100	0
W5	100	0	100	0	100	0
W6	—	—	100	0	—	—
W7A	100	0	100	0	100	0
W7B	100	0	100	0	100	0

Table 9 reports per-operation tail latencies under concurrent execution stress.

Throughput Performance: MongoDB Atlas achieved highest total throughput at 7.58 operations/second (4.93 QPS reads, 2.65 TPS writes), demonstrating 2.6× advantage over Supabase (2.90 ops/sec: 1.89 QPS, 1.02 TPS) and 2.4× over AstraDB (3.18 ops/sec: 2.07 QPS, 1.11 TPS). All databases maintained exact 65.0% read and 35.0% write distribution as specified. MongoDB's throughput dominance reflected consistent sub-50ms median latencies across most operations, while Supabase and AstraDB exhibited higher per-operation latencies that constrained overall throughput under sequential execution. AstraDB marginally outperformed Supabase (3.18 vs. 2.90 ops/sec), attributable to lower write operation latencies (W1, W3, W5 averaging 185–189ms p50) compared to Supabase's 238–258ms baseline. Figure 8 visualizes throughput comparison via grouped bar chart showing Read QPS, Write TPS, and Total throughput per database.

Per-Operation Latency Under Mixed Load: Median per-operation p99 latencies revealed mixed workload amplification effects relative to isolated phases. MongoDB sustained median p99 of 48.70 ms across operations, consistent with A1/A2 isolated read performance (43.84–46.35 ms p99) and Write phase medians (103.91 ms p99), indicating minimal read-write contention. Supabase exhibited median p99 of 563.46 ms, representing 5.8× degradation from A1 warm reads (97.20 ms p99) and 1.2× elevation above Write phase (479.16 ms p99), suggesting resource contention between concurrent read and write transaction processing. AstraDB recorded median p99 of 346.48 ms, falling between A2 random-read performance (211.03 ms p99) and Write phase conditional operations (212.31 ms p99 for normal writes), consistent with eventual consistency model tolerating concurrent operations without strong isolation overhead. Seven operations exceeded the 500ms outlier threshold (Figure 10): Supabase R1 (706ms), R4 (694ms), R5 (1,647ms), W1 (500ms); AstraDB R3 (2,948ms), R6 (742ms), W4 (1,346ms).

Read-Write Interaction Effects: Comparing Mixed phase per-operation latencies to isolated A1/A2 and Write phases revealed workload interaction penalties. Supabase read operations R1–R6 exhibited 2.2–12.1× p99 inflation relative to A1 warm performance (e.g., R5: 1,647ms mixed vs. 136ms A1 warm), indicating PostgreSQL MVCC transaction isolation overhead under concurrent write load. MongoDB read operations showed minimal degradation (R1: 48.70ms mixed vs. 43.49ms A1 warm, 1.12× amplification), while write operations experienced comparable Mixed vs. isolated Write phase latencies (W1: 53.25ms mixed vs. 55.20ms isolated). AstraDB demonstrated variable interaction effects: normal read operations maintained A2-consistent performance (R1: 205ms mixed vs. 226ms A1 warm), but R3 and R5 exhibited extreme tail amplification (2,948ms and 1,577ms respectively) not present in isolated phases, suggesting partition-level hot-spotting under concurrent mixed access. Write operation W4 (1,346ms p99) mirrored isolated Write phase's W3B conditional delete slowness (1,980ms p99), confirming coordinator broadcast overhead persists under mixed workloads. Figure 9 visualizes per-operation p99 distributions for eight normal operations, while outliers are isolated for scale preservation.

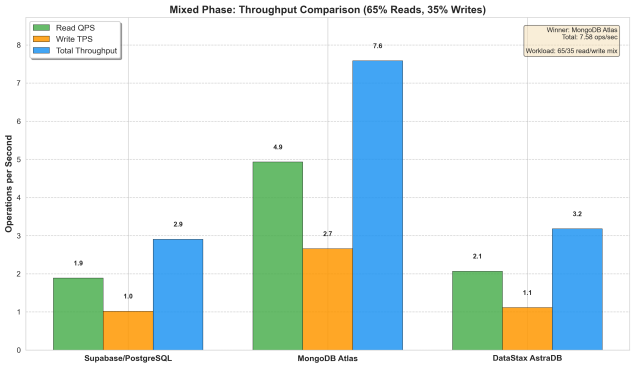

Figure 8: Phase Mixed: Throughput Comparison (Read QPS, Write TPS, Total Operations/Second)

4.6. Summary of Findings

Empirical evaluation across four benchmarking phases revealed distinct performance characteristics for each database system under controlled e-commerce workloads. MongoDB Atlas consistently demonstrated lowest latencies in isolated phases: median p50 of 42.72 ms (A1 fixed-input reads), 42.78 ms (A2 random-input reads), and 55.12 ms

Table 8
Phase Mixed Throughput Metrics (65/35 Workload)

Database	Read QPS	Write TPS	Total Tput	Read %	Write %
Supabase	1.89	1.02	2.90	65.0	35.0
MongoDB	4.93	2.65	7.58	65.0	35.0
AstraDB	2.07	1.11	3.18	65.0	35.0

Table 9
Phase Mixed Per-Operation Tail Latency (p99 ms)

Op	Type	Supa		Mongo		Astra	
		p50	p99	p50	p99	p50	p99
R1	R	318	706	44.3	48.7	188	205
R2	R	239	461	48.6	53.1	184	196
R3	R	278	563	186	189	200	2948
R4	R	336	694	41.1	47.1	198	346
R5	R	280	1647	41.7	45.3	190	1577
R6	R	242	690	40.4	41.4	209	742
W1	W	259	500	43.0	53.3	185	202
W2	W	321	433	43.1	44.7	378	403
W3	W	238	421	43.3	44.9	189	208
W4	W	484	740	450	542	942	1346
W5	W	239	329	54.4	62.0	183	199

Figure 9: Phase Mixed: Per-Operation p99 Latency Distribution

Figure 10: Phase Mixed: Outlier Operation Performance (R1, R3, R4, R5, R6, W1, W4)

(Write phase), alongside highest Mixed phase throughput at 7.58 operations/second. Supabase/PostgreSQL exhibited stable sub-100ms read performance (96.00 ms median p50 in A1, 95.78 ms in A2) with exceptional tail consistency (median amplification 1.01), but suffered 2.2–12.1× latency degradation under Mixed workload concurrent execution. DataStax AstraDB maintained predictable 190–203 ms latencies for normal operations across all phases, yet encountered systematic performance failures on aggregation queries (R6: 413 seconds A1, 386 seconds A2) and conditional operations (W3B: 1.98 seconds), alongside extreme tail amplification under mixed loads (R3: 2.95 seconds p99, R5: 1.58 seconds p99).

All databases achieved 100% write reliability with zero errors across 3,000 total executions. Parameter sensitivity analysis revealed MongoDB and Supabase maintained sub-1ms IQR dispersion (0.84 ms and 0.80 ms median respectively), while AstraDB exhibited 7.9× higher variability (6.63 ms median IQR) reflecting partition-key distribution sensitivity. These quantitative findings establish empirical foundations for architectural interpretation and practical deployment guidance addressed in subsequent discussion.

5. Discussion

5.1. Architectural Trade-offs for Startup Database Selection

The performance evaluation reveals three fundamental tensions governing technology choices for early-stage e-commerce platforms. First, the *speed-consistency trade-off* manifests when comparing MongoDB's throughput advantages against Supabase's transactional guarantees. MongoDB's eventual consistency model enables aggressive optimization strategies yielding superior concurrent performance, making it suitable for user-facing features like product browsing where momentary data staleness remains acceptable. Conversely, Supabase enforces strict ACID compliance through PostgreSQL's MVCC mechanisms, introducing throughput constraints under concurrent loads yet ensuring referential integrity essential for financial workflows—checkout processes, payment reconciliation, and inventory deduction—where atomicity violations would compromise business logic. Startups in pre-revenue validation phases may reasonably prioritize MongoDB's performance characteristics, deferring consistency requirements until transaction volumes justify operational complexity.

Second, *query flexibility versus horizontal scalability* surfaces through AstraDB's partition-key architectural constraints. While Cassandra's distributed design theoretically supports petabyte-scale deployments, realizing this capability demands accepting systematic performance degradation on operations requiring cross-partition coordination, as aggregation queries demonstrated. Early-stage workloads operating within single-digit gigabyte scales gain negligible benefit from distributed architectures while inheriting operational overhead. AstraDB becomes economically justified only post-product-market-fit when write volumes necessitate horizontal scaling and access patterns stabilize

sufficiently to pre-model partition-aligned data relationships—typically 18–24 months into production trajectories.

Third, *developer velocity versus operational optimization* emerges when balancing rapid prototyping against long-term maintainability. MongoDB's schema flexibility accelerates initial development cycles, permitting iterative data model evolution without migration overhead. Supabase's structured schema enforcement introduces upfront design discipline yet prevents technical debt accumulation from unconstrained schema drift. This tension resolves differently across startup lifecycle stages: pre-product-market-fit favors velocity, while post-traction environments benefit from structural constraints preventing scalability bottlenecks.

5.2. Reproducible Framework Contribution

The standardized evaluation methodology developed here addresses critical gaps in existing database assessment practices. Vendor-provided benchmarks optimize for best-case scenarios rather than production-representative workloads reflecting realistic query distributions, cache behavior, and concurrent access patterns. The modular framework—standardized CSV logging, database-agnostic metric computation, automated visualization generation—enables continuous technology reassessment as cloud offerings evolve and organizational requirements shift. This infrastructure reduces benchmarking iteration cycles from weeks to days, democratizing rigorous performance evaluation for engineering teams lacking dedicated data science resources.

5.3. Methodological Boundaries

Several constraints circumscribe findings generalizability. Evaluation targeted single-node free-tier deployments representing early-stage configurations, excluding distributed cluster architectures that materially alter performance at higher transaction volumes. The 500,000-record dataset reflects pre-product-market-fit scale; million-record workloads may exhibit different caching behaviors and index selectivity patterns. Synthetic query generation with uniform random parameters approximates but cannot fully replicate production traffic exhibiting temporal patterns and user behavior clustering influencing cache hit rates. Cost-per-query analysis, operational complexity metrics, and vendor pricing tier variations remain unaddressed despite significance in total ownership calculations. These boundaries establish applicability limits while identifying extensions required for comprehensive deployment guidance.

5.4. Technology Selection Framework for Startups

Empirical findings translate directly into decision criteria for early-stage platforms. MongoDB suits rapid prototyping scenarios where developer velocity and user experience responsiveness outweigh consistency trade-offs. Supabase's ACID guarantees justify throughput penalties for financial transaction processing requiring strict data integrity. AstraDB's architectural constraints preclude adoption until workload patterns stabilize and write volumes justify distributed complexity. The reproducible benchmarking framework enables systematic reassessment as requirements evolve across growth stages, transforming database selection from speculative experimentation into evidence-based architectural decisions aligned with business priorities and resource constraints.

6. Conclusion

6.1. Research Summary

This study addressed the database selection challenge faced by early-stage e-commerce startups by conducting a comprehensive performance benchmarking comparison of three widely adopted cloud-hosted database systems: Supabase/PostgreSQL, MongoDB Atlas, and DataStax AstraDB. A reproducible four-phase testing framework evaluated read performance under fixed and randomized parameter conditions (Phases A1 and A2), isolated write operation efficiency (Write Phase), and realistic concurrent read-write workloads (Mixed Phase) using a synthetic 500,000-record e-commerce dataset spanning customers, products, orders, and order items. All benchmarking operations employed phase-specific index strategies, structured CSV logging for metric reproducibility, and automated Python-based data processing pipelines generating percentile distributions, dispersion indicators, throughput metrics, and publication-quality visualizations across 10 comparative figures and 8 cross-database summary tables.

6.2. Key Findings Recap

Empirical benchmarking revealed distinct performance profiles across databases: MongoDB Atlas consistently achieved lowest latencies in isolated operations (median p50 of 42.72 ms for fixed-input reads, 42.78 ms for random-input reads, 55.12 ms for write operations) and dominated realistic workload throughput at 7.58 operations/second under 65/35 read-write mix, demonstrating 2.6× advantage over competitors; Supabase/PostgreSQL exhibited exceptional tail consistency with median tail amplification ratio of 1.01 and tightest parameter sensitivity control (median IQR of 0.80 ms), alongside dramatic cache effectiveness evidenced by 97.68% cold-to-warm latency reduction on aggregation queries, but suffered severe throughput degradation (2.2–12.1× latency inflation) under concurrent mixed workloads attributable to transaction isolation overhead; DataStax AstraDB maintained predictable 190–203 ms baseline latencies for normal operations but encountered systematic performance failures on aggregation queries (R6: 413 seconds in A1, 386 seconds in A2) and conditional operations (W3B: 1.98 seconds), alongside 7.9× higher parameter variability (median IQR 6.63 ms) reflecting partition-key sensitivity, with extreme tail amplification under mixed loads (R3: 2,948 ms, R5: 1,577 ms). All three databases achieved 100% write reliability across 3,000 total executions with zero error occurrences.

6.3. Practical Recommendations

For early-stage e-commerce startups prioritizing rapid development and operational simplicity, MongoDB Atlas

emerges as the optimal general-purpose choice, delivering sub-50ms read latencies across all query complexities, 7.58 operations/second realistic workload throughput, and minimal performance degradation under concurrent read-write scenarios. Startups building latency-critical features—product search APIs, real-time inventory checks, or sub-100ms recommendation engines—should adopt MongoDB's document-oriented model, which eliminates JOIN overhead and maintains consistent performance even under parameter randomization (42.78 ms median p50 in A2 versus 42.72 ms in A1 fixed-input conditions). However, this speed advantage trades eventual consistency guarantees for performance; applications requiring strict ACID transactions (multi-step checkout flows, financial reconciliation, referential integrity enforcement) should instead select Supabase/PostgreSQL, which demonstrated exceptional tail consistency (1.01 median amplification ratio) and reliable transactional semantics, despite 2.6× lower throughput (2.90 ops/sec) and vulnerability to concurrent workload contention. The 97.68% cache effectiveness on aggregation queries further positions Supabase favorably for analytics-heavy dashboards or reporting workloads where query result caching provides substantial performance uplift.

DataStax AstraDB addresses future horizontal scaling requirements but introduces critical query flexibility constraints unsuitable for early-stage exploration. The 413-second aggregation failure (Query R6) and 1.98-second conditional delete latency (W3B) reflect fundamental architectural limitations in Cassandra's partition-key-centric design, requiring upfront data modeling expertise that most startups lack during rapid iteration phases. AstraDB becomes viable only after product-market fit when workload patterns stabilize and horizontal write scalability justifies accepting query restrictions—startups anticipating >10,000 writes/second or multi-region active-active replication should evaluate AstraDB at that growth inflection point. For hybrid deployment strategies, startups can adopt polyglot persistence: MongoDB for primary transactional workloads, Supabase for admin dashboards requiring complex JOINs, and eventual migration of high-volume write paths to AstraDB once scale demands justify operational complexity. Trade-off priorities crystallize as: MongoDB optimizes for speed and developer velocity; Supabase prioritizes data correctness and SQL familiarity; AstraDB targets extreme write scalability at the cost of query expressiveness. Early-stage teams constrained by limited engineering resources should default to MongoDB's balanced performance profile, deferring architectural diversification until scaling bottlenecks empirically justify the added complexity.

6.4. Methodological Contribution

Beyond empirical findings, this research establishes a reproducible benchmarking framework enabling technology teams to conduct rigorous database evaluations without specialized performance engineering expertise. All benchmarking components—query execution scripts with phase-specific CSV logging, four standardized data processing pipelines computing percentile distributions and dispersion metrics, and automated figure generation tools producing publication-quality visualizations—are fully documented with explicit input/output specifications and modular architecture. Researchers and practitioners can adapt this framework to evaluate alternative database systems (Firebase, CockroachDB, TimescaleDB) by maintaining the standardized CSV schema (columns: db_system, query_id, execution_ms, error_code, timestamp, phase, is_cold) while substituting database-specific connection and query execution logic. The eight-cell Jupyter notebook structure (imports → configuration → data loading → validation → primary metrics → derived metrics → output → summary) ensures consistent metric computation across heterogeneous systems, while outlier detection thresholds (10× median p50, 5× median p99) and adaptive visualization scaling prevent performance anomalies from distorting comparative analysis. This methodology reduces benchmarking setup time from weeks to days, democratizing performance evaluation for resource-constrained startups lacking dedicated infrastructure teams, and establishes empirical foundations for evidence-based technology adoption decisions in early-stage product development contexts.

6.5. Closing Statement

By combining rigorous empirical evaluation with an accessible, reproducible methodology, this research transforms database selection from speculative trial-and-error into evidence-based technology adoption, enabling early-stage e-commerce startups to allocate scarce engineering resources toward product differentiation rather than infrastructure experimentation. The quantitative performance profiles, scenario-based recommendations, and reusable benchmarking framework collectively reduce the technical risk inherent in foundational architecture decisions during critical early-stage growth phases when database migration costs would otherwise jeopardize product-market-fit timelines.

References

Akther, F., 2023. E-commerce in India: trends, hurdles, and growth opportunities. Formosa Journal of Science and Technology 2, 2871–2880.

Anonymous, 2023. A study on user behavior of e-commerce platforms based on data analysis. Journal of Global Humanities and Social Sciences 4, 181–185.

Battle, L., Eichmann, P., Angelini, M., Catarci, T., Santucci, G., Zheng, Y., et al., 2020. Database benchmarking for supporting real-time interactive querying of large data, in: Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data, pp. 1571–1587.

Baye, M.R., De los Santos, B., Wildenbeest, M.R., 2012. The evolution of product search. Journal of Law, Economics, and Policy 9, 201.

Bhatia-Kalluri, A., 2021. E-commerce for rural micro-entrepreneurs: Mapping restrictions, ecologies of use and trends for development. ArXiv preprint arXiv:2108.09759.

Blagec, K., Kraiger, J., Frühwirt, W., Samwald, M., 2023. Benchmark datasets driving artificial intelligence development fail to capture the needs of medical professionals. Journal of Biomedical Informatics 137, 104274.

Boncz, P., Neumann, T., Erling, O., 2013. TPC-H analyzed: Hidden messages and lessons learned from an influential benchmark, in: Technology

Conference on Performance Evaluation and Benchmarking, Springer International Publishing, Cham. pp. 61–76.

Cole, R., Funke, F., Giakoumakis, L., Guy, W., Kemper, A., Krompass, S., et al., 2011. The mixed workload CH-benCHmark, in: Proceedings of the Fourth International Workshop on Testing Database Systems, pp. 1–6.

Cooper, B.F., Silberstein, A., Tam, E., Ramakrishnan, R., Sears, R., 2010. Benchmarking cloud serving systems with YCSB, in: Proceedings of the 1st ACM symposium on Cloud computing, pp. 143–154.

Difallah, D.E., Pavlo, A., Curino, C., Cudre-Mauroux, P., 2013. OLTP-bench: An extensible testbed for benchmarking relational databases. Proceedings of the VLDB Endowment 7, 277–288.

Dodge, R.C., Menascé, D.A., Barbára, D., 2001. Testing e-commerce site scalability with TPC-W, in: Proceedings of 2001 Computer Measurement Group Conference.

Erling, O., Averbuch, A., Larriba-Pey, J., Chafi, H., Gubichev, A., Prat, A., et al., 2015. The LDBC social network benchmark: Interactive workload, in: Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, pp. 619–630.

Funke, F., Kemper, A., Neumann, T., 2011. Benchmarking hybrid OLTP&OLAP database systems, in: Datenbanksysteme für Business, Technologie und Web (BTW), Gesellschaft für Informatik eV. pp. 390–409.

Iosup, A., Musaafir, A., Uta, A., Pérez, A.P., Szárnyas, G., Chafi, H., et al., 2020. The LDBC Graphalytics benchmark. ArXiv preprint arXiv:2011.15028.

Kaszó, M., Levendovszky, T., . TPC Benchmark.

Kavalanekar, S., Narayanan, D., Sankar, S., Thereska, E., Vaid, K., Worthington, B., 2009. Measuring database performance in online services: a trace-based approach, in: Technology Conference on Performance Evaluation and Benchmarking, Springer Berlin Heidelberg, Berlin, Heidelberg. pp. 132–145.

Khouja, M., Kamoun, F., Lladó, C.M., Puigjaner, R., 2010. Experimenting With the TPC-W E-commerce Benchmark.

Lee, S., Kim, J.D., 2024. When do startups scale? Large-scale evidence from job postings. Strategic Management Journal 45, 1633–1669.

Liyanage, D., Pandey, S., Goldstein, J., Cahill, M., Dey, A., Fekete, A., Röhm, U., 2025. A Benchmark for Databases with Varying Value Lengths. ArXiv preprint arXiv:2508.07551.

Longjohn, R., Kelly, M., Singh, S., Smyth, P., 2024. Benchmark data repositories for better benchmarking, in: Advances in Neural Information Processing Systems, pp. 86435–86457.

Menascé, D.A., 2002. TPC-W: A benchmark for e-commerce. IEEE Internet Computing 6, 83–87.

Microsoft Corporation, 2023. How to Use Batching to Improve Application Performance. Azure SQL Database Documentation, Microsoft Learn. https://learn.microsoft.com/en-us/azure/azure-sql/performance-improve-use-batching.

Nah, F.F.H., Hong, W., Chen, L., Lee, H.H., 2012. Information search patterns in e-commerce product comparison services, in: Cross-Disciplinary Models and Applications of Database Management: Advancing Approaches. IGI Global Scientific Publishing, pp. 131–145.

Panpaliya, T., 2020. Benchmarking MongoDB multi-document transactions in a sharded cluster.

Patil, S., Polte, M., Ren, K., Tantisiriroj, W., Xiao, L., López, J., et al., 2011. YCSB++ benchmarking and performance debugging advanced features in scalable table stores, in: Proceedings of the 2nd ACM Symposium on Cloud Computing, pp. 1–14.

Pena-Ortiz, R., Gil, J.A., Sahuquillo, J., Pont, A., 2012. Providing TPC-W with web user dynamic behavior. CLEI Electronic Journal 15, 2–2.

Petzka, H., Stadler, C., Katsimpras, G., Haarmann, B., Lehmann, J., 2017. Benchmarking faceted browsing capabilities of triplestores, in: Proceedings of the 13th International Conference on Semantic Systems, pp. 128–135.

Püroja, D., Waudby, J., Boncz, P., Szárnyas, G., 2023. The LDBC Social Network Benchmark Interactive workload v2: A transactional graph query benchmark with deep delete operations, in: Technology Conference on Performance Evaluation and Benchmarking, Springer Nature Switzerland, Cham. pp. 107–123.

Qi, S., Tong, B., Hu, J., Lin, H., Pang, Y., Yuan, W., et al., 2025. The LDBC Financial Benchmark: Transaction Workload. Proceedings of the VLDB Endowment 18, 3007–3020.

Ruben, B.E.S., 2021. Benchmarking Distributed Database Performance and Dependability under Partial System Failures. Ph.D. thesis. Delft University of Technology.

Shih, D.H., Lu, K.C., Shih, P.Y., 2019. Exploring shopper's browsing behavior and attention level with an EEG biosensor cap. Brain Science 9, 301.

Smith, W.D., 2000. TPC-W: Benchmarking an ecommerce solution.

Song, I.Y., Whang, K.Y., 2000. Database design for real-world e-commerce systems. IEEE Data Engineering Bulletin 23, 23–28.

Springtail, . Startup Database Trends Report. https://www.springtail.io/startup-database-trends.

Szárnyas, G., Bebee, B., Birler, A., Deutsch, A., Fletcher, G., Gabb, H.A., et al., 2023. The linked data benchmark council (LDBC): Driving competition and collaboration in the graph data management space, in: Technology Conference on Performance Evaluation and Benchmarking, Springer Nature Switzerland, Cham. pp. 90–106.

Szárnyas, G., Waudby, J., Steer, B.A., Szakállas, D., Birler, A., Wu, M., et al., 2022. The LDBC social network benchmark: Business intelligence workload. Proceedings of the VLDB Endowment 16, 877–890.

Thete, B.B., 2024. E-Commerce in India – Issues and Opportunities. International Journal of Research and Publication Review 5, 1643–1649.

TiDB Team, 2025. Evaluating Free Cloud Databases. https://www.pingcap.com/article/evaluating-free-cloud-databases-key-features-tidb-serverless/.

Tözün, P., Pandis, I., Kaynak, C., Jevdjic, D., Ailamaki, A., 2013. From A to E: analyzing TPC's OLTP benchmarks: the obsolete, the ubiquitous, the unexplored, in: Proceedings of the 16th International Conference on Extending Database Technology, pp. 17–28.

Transaction Processing Performance Council, 2014. TPC-H Benchmark Revision 2.17.1. Technical Report.

Wang, J., Xiao, K., Sun, Q., Zhao, H., Luo, T., Zhang, J., Zeng, X., 2025. ShoppingBench: A Real-World Intent-Grounded Shopping Benchmark for LLM-based Agents. ArXiv preprint arXiv:2508.04266.

CRediT authorship contribution statement

Avyaansh Dixit: Conceptualization, Methodology, Formal analysis, Visualization. **Nidhi Singh:** Project administration, Supervision, Validation, Writing – original draft. **Deepinder Kaur:** Resources, Data curation. **Aryan:** Investigation, Software, Data curation. **Gargi Singh:** Writing – review and editing.